

ADR-001 · Embeddings Sourced From the Hugging Face Inference Router, Not the Local Custom LLM Endpoint

Field	Detail
Decision	Use <code>langchain_huggingface.HuggingFaceEndpointEmbeddings</code> (HF Inference Providers router) for all embedding calls in <code>3_rag_v1.py</code> , rather than routing embeddings through the project's local <code>CUSTOM_API_BASE</code> chat server
Date	2026-07-08
Context	The local server only implements <code>/v1/chat/completions</code> , <code>/v1/completions</code> , and <code>/health</code> (confirmed via its <code>/openapi.json</code> , see Bugs.md BUG-001) — it has no embeddings route under any configuration. An embeddings client was needed that didn't depend on standing up a second local server or model.
Options Considered	Add an embeddings endpoint to the local TGI-style server (out of scope, external dependency) · fall back to <code>OpenAIEmbeddings</code> against the real OpenAI API (works, but reintroduces the cloud cost/key dependency the local-LLM setup was meant to avoid) · <code>HuggingFaceEndpointEmbeddings</code> via the HF Inference router (free tier, <code>HF_TOKEN</code> already available, confirmed working via a direct <code>curl</code> test)
Chosen Solution	<code>HuggingFaceEndpointEmbeddings(model="sentence-transformers/all-MiniLM-L6-v2", provider="hf-inference", huggingfacehub_api_token=HF_TOKEN)</code> , model name overridable via <code>HF_EMBEDDING_MODEL_NAME</code>
Rationale	Reuses credentials/infrastructure already present in <code>.env</code> rather than adding a new provider; keeps the chat LLM fully local while sourcing the one capability (embeddings) the local server can't provide from a free, already-available router.
Impact	<code>3_rag_v1.py</code> embeddings block rewritten; <code>langchain-huggingface</code> added as a dependency, pinned <code><1.0.0</code> for <code>langchain-core</code> compatibility. Chat generation (<code>llm_setup.py</code>) is unaffected. Note: <code>3_rag_v4.py</code> still uses <code>OpenAIEmbeddings</code> and has not been migrated under this decision — see Architecture.md module breakdown for <code>3_rag_v4.py</code> .

ADR-002 · Project .venv Built on Python 3.12, Not 3.14

Field	Detail
Decision	Create the project virtual environment with Python 3.12 (<code>py -3.12 -m venv myenv</code>) rather than the newer 3.14
Date	2026-07-08
Context	<code>python3</code> / <code>python3.11</code> were not runnable at all — no real Python was on <code>PATH</code> (Windows Store alias redirect) and no 3.11 runtime was registered with the <code>py</code> launcher (<code>py -3.11 --version</code> → "No runtime installed that matches 3.11"). <code>py --list</code> / <code>py -0</code> showed only two real installed runtimes: 3.14.3 and 3.12.10.
Options Considered	Python 3.14 (newest installed, but very recently released — higher risk of dependency wheels/compatibility gaps with the LangChain/LangSmith stack) · Python 3.12 (installed, one minor version behind, broader ecosystem/wheel support at time of setup) · install Python 3.11 via <code>py install 3.11</code> (matches what was originally attempted, but adds an extra install step for no clear benefit over 3.12)
Chosen Solution	<code>py -3.12 -m venv myenv</code>
Rationale	3.12 offered the best balance of "already installed" and "broadly compatible with LangChain/LangSmith packages" without gambling on a same-week-class Python release (3.14) having full wheel support from every dependency.
Impact	<code>.venv</code> in the project root targets Python 3.12.10. No code depends on 3.12-specific syntax; revisiting this choice later (e.g. moving to 3.14) should be low-risk if a concrete reason arises.

ADR-003 · Remaining OpenAI-Dependent Scripts Migrated Onto the Local TGI Proxy + HF Router Stack

Field	Detail
Decision	Migrate <code>3_rag_v4.py</code> (embeddings + chat), <code>4_agent.py</code> (chat), and <code>5_langgraph.py</code> (chat) off the OpenAI API entirely, onto the same <code>llm_setup.llm</code> (local TGI Proxy) + <code>HuggingFaceEndpointEmbeddings</code> (HF Inference router) pattern already used by <code>3_rag_v1-v3</code> (ADR-001)
Date	2026-07-09
Context	The user's OpenAI API key stopped working. <code>.env</code> was intentionally trimmed to drop <code>OPENAI_API_KEY</code> (along with <code>GROQ_*</code> and several other unused vars) as part of a cleanup pass, which meant <code>3_rag_v4.py</code> (<code>OpenAIEmbeddings</code> + bare <code>ChatOpenAI("gpt-4o-mini")</code>), <code>4_agent.py</code> (bare <code>ChatOpenAI()</code>), and <code>5_langgraph.py</code> (bare <code>ChatOpenAI("gpt-4o-mini")</code>) would fail outright — they were the only scripts still depending on OpenAI credentials.
Options Considered	Restore <code>OPENAI_API_KEY</code> and keep troubleshooting the account/billing issue (rejected — key is confirmed non-functional and out of this project's control) · leave the three scripts broken/unmaintained (rejected — they're active teaching demos) · migrate them onto the already-proven local-LLM + HF-embeddings stack (chosen)
Chosen Solution	<code>3_rag_v4.py</code> : <code>OpenAIEmbeddings</code> → <code>HuggingFaceEndpointEmbeddings</code> (default <code>sentence-transformers/all-MiniLM-L6-v2</code>), <code>ChatOpenAI</code> → <code>llm</code> from <code>llm_setup</code> . <code>4_agent.py</code> : <code>ChatOpenAI()</code> → <code>llm</code> from <code>llm_setup</code> . <code>5_langgraph.py</code> : <code>ChatOpenAI(...)</code> → <code>llm</code> from <code>llm_setup</code> ; with <code>structured output</code> replaced with a prompt + <code>PydanticOutputParser</code> approach (forced by BUG-002, discovered while testing this migration).
Rationale	Keeps the whole project on one credential set (<code>CUSTOM_API_BASE/CUSTOM_API_KEY/HF_TOKEN</code>) instead of two, removes a dependency on a currently-broken external account, and reuses a pattern already validated end-to-end in <code>3_rag_v1.py</code> . Verified each migrated script actually runs against the local stack before considering this done (see <code>Status.md</code> 2026-07-09 entry).

Impact	3_rag_v4.py, 4_agent.py, 5_langgraph.py rewritten; dead OpenAIEmbeddings/ChatOpenAI imports also removed from 3_rag_v2.py/3_rag_v3.py. OPENAI_API_KEY is no longer required anywhere in this project. Supersedes the "not yet migrated" note this project's Architecture.md previously carried for 3_rag_v4.py. Related: BUG-002, BUG-003.
---------------	--

ADR-004 · Tracking Docs Moved Into docs/ and No Longer Git-Ignored

Field	Detail
Decision	Move Status.md, Architecture.md, Decisions.md, Research.md, Bugs.md (plus the course transcript and the tracking guide) into a docs/ folder, and remove the .gitignore rule that had excluded the five tracking files from version control
Date	2026-07-09
Context	The tracking system was originally set up mirroring RAG-work's convention of keeping these five files git-ignored (local-only, never committed). The user restructured .gitignore (also dropping an unrelated Graphify/Claude-specific section) and relocated the docs into docs/ without re-adding that exclusion.
Options Considered	Keep the tracking docs git-ignored/local-only, matching RAG-work (rejected — no longer what .gitignore reflects) · commit docs/ as part of the repo so history/decisions/bugs travel with the code and are visible in PRs (chosen, matches the user's actual .gitignore edit)
Chosen Solution	docs/ is a normal, tracked folder; no gitignore rule excludes it or the five tracking files
Rationale	Reflects the change the user already made rather than fighting it; since .env (the only place secrets live) remains git-ignored, committing the docs doesn't create a credential-leak risk — redacted references like CUSTOM_API_BASE/HF_TOKEN (env var names, not values) are the only credential-adjacent content in these files.
Impact	Future commits will include docs/*.md changes; anyone cloning the repo gets the full Status/Architecture/Decisions/Research/Bugs history. No code impact.